中 南 大 学

实 验 指 导 书

课程名称	信息内容安全
实验学时	2
适用专业	信息安全、计算机
指导老师	李芳芳教授

实验 1 中文文本分词与预处理

实验环境： Python

实验目的：

1、理解中文文本分词的基本概念和流程；

2、掌握 Jieba 分词工具的安装与基本使用方法；

3、掌握 Jieba 分词的三种模式：精确模式、全模式、搜索引擎模式；

4、掌握使用 Jieba 进行词性标注的方法；

5、能够对分词结果进行停用词过滤、词频统计等预处理操作。

实验内容：

阅读并运行指导书所附程序代码，把实现以下功能的相关代码摘录出来：

1、使用 Jieba 对自定义句子进行精确模式分词；

2、使用 Jieba 对自定义句子进行全模式分词；

3、使用 Jieba 对自定义句子进行搜索引擎模式分词；

4、使用 Jieba 进行带词性标注的分词；

5、读取《三国演义》文本，进行分词、停用词过滤，统计词频并输出前 20 个高频词；

6、统计《三国演义》中主要人物（曹操、刘备、诸葛亮、关羽、张飞等）的出场次数。

实验要求：

(1) 根据实验报告模板详细书写实验报告，在实验报告中给出实现以上每个功能的相应代码。

(2) 在可视化教学平台上提交实验报告。实验报告命名为：学号+姓名+实验报告 1.doc。

附：分词及预处理相关代码

```
import jieba
import jieba.posseg as pseg


def precise_seg(s):
    return jieba.lcut(s, cut_all=False)


def full_seg(s):
    return jieba.lcut(s, cut_all=True)


def search_seg(s):
    return jieba.lcut_for_search(s)


def pos_seg(s):
    return pseg.lcut(s)


def top20_words():
    txt = open('data/threekingdoms.txt', 'r', encoding='utf-8').read()
    words = jieba.lcut(txt)
    counts = {}
    stopwords = {'将军', '却说', '荆州', '二人', '不可', '不能', '如此', '今日',
'次日',
                 '于是', '如何', '主公', '一人', '不敢', '大喜', '陛下', '不知',
'汉中',
                 '只见', '都督', '后主'}
    for w in words:
        if len(w) == 1:
            continue
        if w in stopwords:
            continue
        # 人物别名归一化
        if w == '孟德' or w == '丞相':
            w = '曹操'
        elif w == '玄德' or w == '玄德曰':
            w = '刘备'
        elif w == '孔明' or w == '孔明曰':
            w = '诸葛亮'
        elif w == '关公' or w == '云长':
            w = '关羽'
        counts[w] = counts.get(w, 0) + 1
    li = list(counts.items())
    li.sort(key=lambda x: x[1], reverse=True)
    print('三国演义前 20 高频词：')
    for i in range(20):
        print('{:<3}{:<8}{:>6}'.format(i+1, li[i][0], li[i][1]))
```

```
def people_count():
    txt = open('data/threekingdoms.txt', 'r', encoding='utf-8').read()
    words = jieba.lcut(txt)
    counts = {}
    people = ['曹操', '诸葛亮', '刘备', '关羽', '张飞', '吕布', '赵云', '姜维', '
周瑜',
              '孙权', '司马懿', '黄忠', '马超', '魏延', '庞统', '袁绍']
    for w in words:
        if len(w) == 1:
            continue
        if w == '孟德' or w == '丞相':
            w = '曹操'
        elif w == '玄德' or w == '玄德曰':
            w = '刘备'
        elif w == '孔明' or w == '孔明曰':
            w = '诸葛亮'
        elif w == '关公' or w == '云长':
            w = '关羽'
        counts[w] = counts.get(w, 0) + 1
    result = {k: v for k, v in counts.items() if k in people}
    li = list(result.items())
    li.sort(key=lambda x: x[1], reverse=True)
    print('主要人物出场次数：')
    for item in li:
        print('{:<8}{:>6}'.format(item[0], item[1]))

if __name__ == '__main__':
    s = '中南大学信息内容安全课程实验'
    print('精确模式：', precise_seg(s))
    print('全模式：', full_seg(s))
    print('搜索模式：', search_seg(s))
    print('词性标注：', pos_seg(s))
    top20_words()
    people_count()
```

实验步骤：

1、运行开发环境平台 PyCharm，进入实验环境。

2、新建 Python 项目，填入项目名称及存储位置，确定后建立项目。

3、创建 Python 文件。

4、输入以上所附代码。

5、通过菜单"Run"下的"Run"命令运行程序。

6、查看运行结果，并对比程序，找出相应功能的代码。

7、将代码及运行结果填写到实验报告中。

实验 2 文本向量化表示

实验环境： Python

实验目的：

1、理解文本向量化的基本概念和意义；

2、掌握词袋模型（Bag of Words）的原理及 CountVectorizer 的使用；

3、掌握 TF-IDF 加权方法的原理及 TfidfTransformer 的使用；

4、能够将中文文本经过分词后转换为向量矩阵；

5、理解并比较不同向量化方法的优缺点。

实验内容：

阅读并运行指导书所附程序代码，把实现以下功能的相关代码摘录出来：

1、使用 CountVectorizer 将分词后的文本转换为词频向量（词袋模型）；

2、使用 TfidfTransformer 将词频向量转换为 TF-IDF 权重向量；

3、使用 TfidfVectorizer 一步完成文本到 TF-IDF 向量的转换；

4、对新闻文本数据集进行分词、去停用词、TF-IDF 向量化处理；

5、输出特征词列表及对应的 TF-IDF 权重矩阵。

实验要求：

(1) 根据实验报告模板详细书写实验报告，在实验报告中给出实现以上每个功能的相应代码。

(2) 在可视化教学平台上提交实验报告。实验报告命名为：学号+姓名+实验报告 2.doc。

附：文本向量化相关代码

```
import jieba
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

def load_stopwords(path):
    with open(path, 'r', encoding='utf-8') as f:
        return [line.strip() for line in f]

def preprocess_text(text, stopwords):
    words = jieba.lcut(text.strip())
    return ' '.join([w for w in words if w not in stopwords and len(w) > 1])

def bow_vectorize(corpus):
    # 词袋模型：CountVectorizer
    vectorizer = CountVectorizer()
    X = vectorizer.fit_transform(corpus)
    print('词袋模型特征词：', vectorizer.get_feature_names_out())
    print('词频矩阵：', X.toarray())
    return X, vectorizer

def tfidf_transform(X_count):
    # TF-IDF 转换
    transformer = TfidfTransformer()
    X_tfidf = transformer.fit_transform(X_count)
    print('TF-IDF 权重矩阵：')
    print(np.round(X_tfidf.toarray(), 4))
    return X_tfidf

def tfidf_vectorize(corpus):
    # 一步完成 TF-IDF 向量化
    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform(corpus)
    print('特征维度：', X.shape)
    print('特征词：', vectorizer.get_feature_names_out()[:20])
    return X, vectorizer

def news_vectorization(corpus_path, stopwords_path):
    stopwords = load_stopwords(stopwords_path)
    corpus = []
    with open(corpus_path, 'r', encoding='utf-8') as f:
        for line in f:
            corpus.append(preprocess_text(line, stopwords))
    # TF-IDF 向量化
    vectorizer = TfidfVectorizer(max_features=5000)
```

```
    X = vectorizer.fit_transform(corpus)
    print('文档数：', X.shape[0])
    print('特征数：', X.shape[1])
    print('前 10 个特征词：', vectorizer.get_feature_names_out()[:10])
    return X, vectorizer


if __name__ == '__main__':
    sample_corpus = [
        '这是一个测试文档',
        '这是第二个测试文档',
        '文档向量化是文本处理的重要步骤'
    ]
    # 示例 1：词袋模型
    X_count, vec = bow_vectorize(sample_corpus)
    # 示例 2：TF-IDF 转换
    tfidf_transform(X_count)
    # 示例 3：一步 TF-IDF
    tfidf_vectorize(sample_corpus)
    # 示例 4：新闻文本向量化（需准备数据文件）
    # news_vectorization('data/news.txt', 'data/stop_words.txt')
```

实验步骤：

1、运行开发环境平台 PyCharm，进入实验环境。

2、新建 Python 项目，填入项目名称及存储位置，确定后建立项目。

3、创建 Python 文件。

4、输入以上所附代码。

5、通过菜单"Run"下的"Run"命令运行程序。

6、查看运行结果，并对比程序，找出相应功能的代码。

7、将代码及运行结果填写到实验报告中。